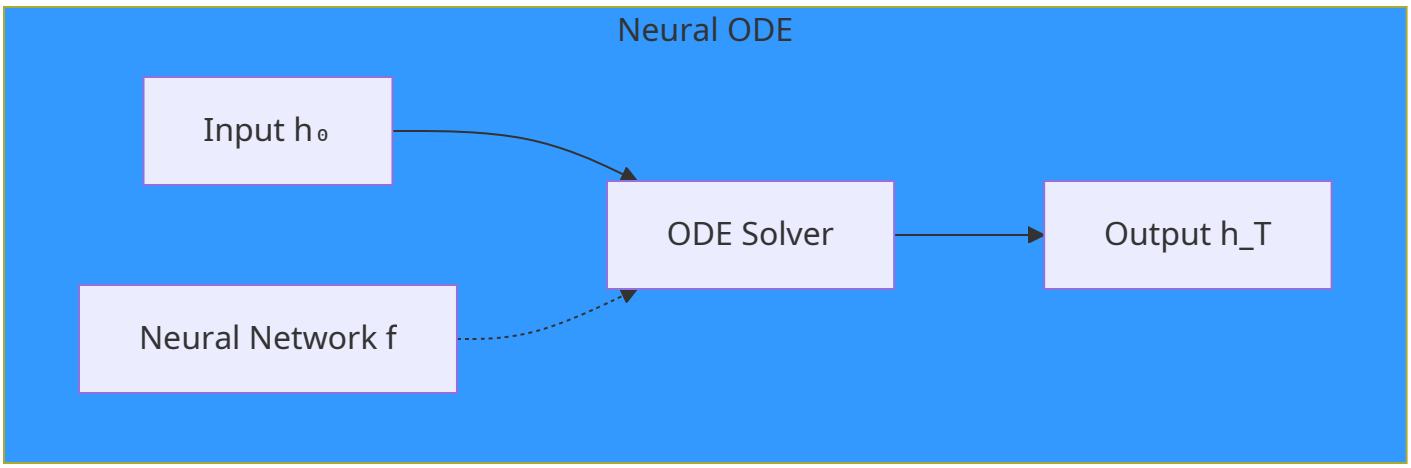
Neural Ordinary Differential Equations

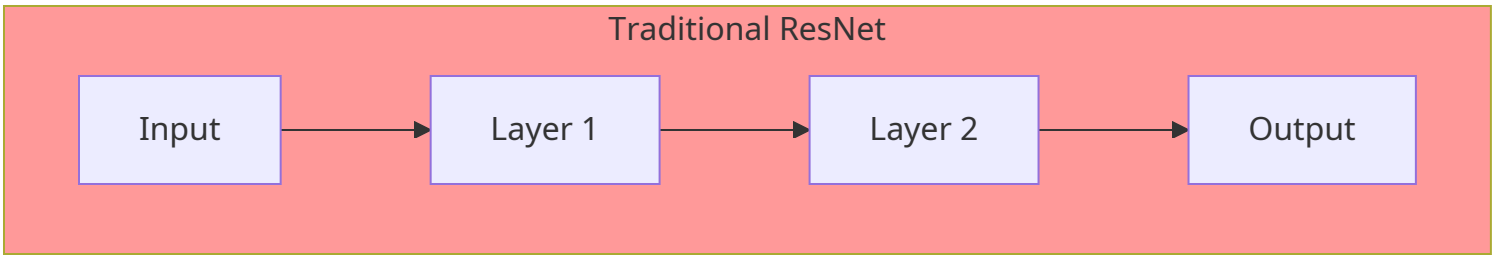
Info

Neural ODEs replace discrete layers with continuous dynamics. Instead of stacking layers, you define the derivative of the hidden state with a neural network and let an ODE solver compute the output.

Overview

Neural ODEs are a family of deep neural network models that replace discrete layers with continuous dynamics. Instead of specifying a sequence of hidden layers, we parameterize the **derivative of the hidden state** using a neural network, then use a standard ODE solver to compute the output.





Intuition

Think of ResNet as climbing stairs (discrete steps) and Neural ODE as sliding down a smooth ramp (continuous path). The ramp needs less memory and adapts to your speed!

Key Idea: From Discrete to Continuous

Traditional Residual Network

A standard ResNet defines a **discrete** sequence of transformations:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t)$$

where $t \in \{0, 1, \ldots, T\}$ represents the layer index.

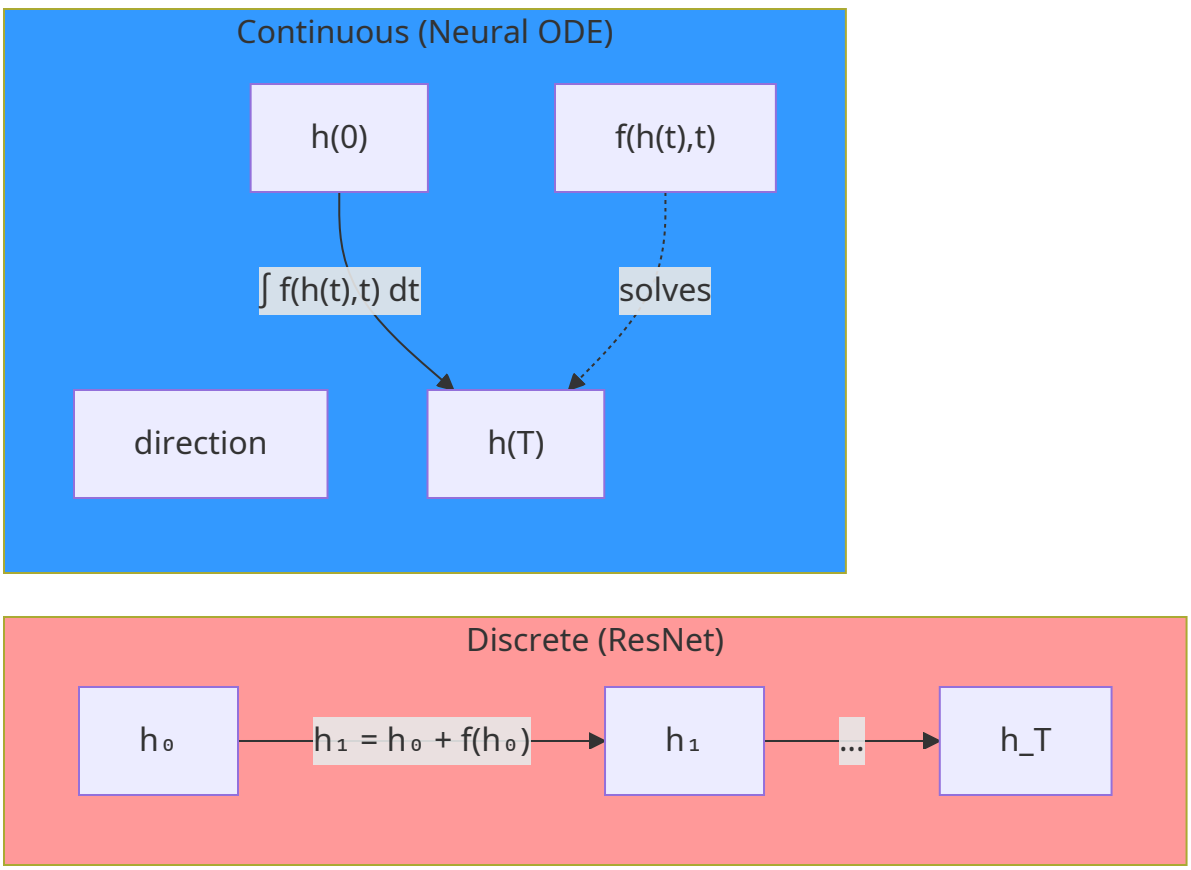
Neural ODE (Continuous Limit)

As we add more layers and take smaller steps, this becomes a **continuous** transformation:

$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta)$$

where:

- $\mathbf{h}(t)$ is the hidden state at continuous "time" t
- f is a neural network that predicts the **derivative** of the hidden state
- θ are the network parameters



Mathematical Formulation

Forward Pass (Solving the ODE)

Given:

- Initial state $\mathbf{h}(t_0)$ (the input)
- Neural network f that defines the dynamics
- Integration time T

The output is computed by solving the **initial value problem**:

$$\mathbf{h}(T) = \mathbf{h}(t_0) + \int_{t_0}^{T} f(\mathbf{h}(t), t, \theta)\, dt$$

In practice, we use a black-box ODE solver:

$$\mathbf{h}(T) = \mathrm{ODESolve}(\mathbf{h}(t_0), f, t_0, T, \theta)$$

> **Implementation Note**
>
> Use `torchdiffeq.odeint` which handles adaptive stepping and backpropagation automatically.

Key Properties

Property	Explanation
Constant Memory	O(1) memory cost - no need to store intermediate activations
Adaptive Computation	Solver adjusts step size based on problem complexity
Parameter Efficiency	Parameters are shared across all "layers"

Backpropagation: The Adjoint Method

> **The Challenge**
>
> You can't just backprop through the ODE solver directly - it would be memory-intensive and numerically unstable. The adjoint method solves this!

The biggest technical challenge is computing gradients through the ODE solver. The **adjoint sensitivity method** solves this elegantly.

The Adjoint State

Define the **adjoint** as the gradient of the loss with respect to the hidden state:

$$\mathbf{a}(t) = \frac{\partial L}{\partial \mathbf{h}(t)}$$

Dynamics of the Adjoint

The adjoint itself follows an ODE:

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f(\mathbf{h}(t), t, \theta)}{\partial \mathbf{h}}$$

This is derived from the continuous chain rule - it tells us how the sensitivity flows backward in time.

Why this is better

Instead of storing all intermediate states (which would be O(T) memory), we solve another ODE backward in time. This gives us **O(1) memory** and the ability to trade accuracy for speed!

Computing Gradients

To get gradients with respect to parameters:

$$\frac{dL}{d\theta} = -\int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f(\mathbf{h}(t), t, \theta)}{\partial \theta} \, dt$$

Paper Results

On MNIST classification:

- **ResNet**: 0.41% error, 0.60M params
- **Neural ODE**: 0.42% error, 0.22M params
- Same performance with **1/3 the parameters**!

The Four Key Equations

Forward Pass

$$\mathbf{h}(T) = \text{ODESolve}(\mathbf{h}(t_0), f, t_0, T, \theta)$$

Adjoint Dynamics

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^\top \frac{\partial f}{\partial \mathbf{h}}$$

Parameter Gradients

$$\frac{dL}{d\theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^\top \frac{\partial f}{\partial \theta}\, dt$$

Continuous Change of Variables

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr}\left(\frac{\partial f}{\partial \mathbf{z}(t)}\right)$$

Resources

- **Original Paper**: Neural Ordinary Differential Equations (NeurIPS 2018)
- **Code**: torchdiffeq GitHub Repository

Lotka - Volterra System

Note

Learning and testing how **Neural ODEs** work by modeling a classical **two-variable differential equation system**: the Lotka–Volterra predator-prey dynamics.

Explaining the Lotka–Volterra System

The **Lotka–Volterra system** models the interaction between two species:

- Prey (rabbits)
- Predator (wolves)

The idea is simple:

- Rabbits grow naturally if left alone.

- Wolves depend on rabbits for food.
- Interaction between them creates oscillatory population behavior.

Differential Equations

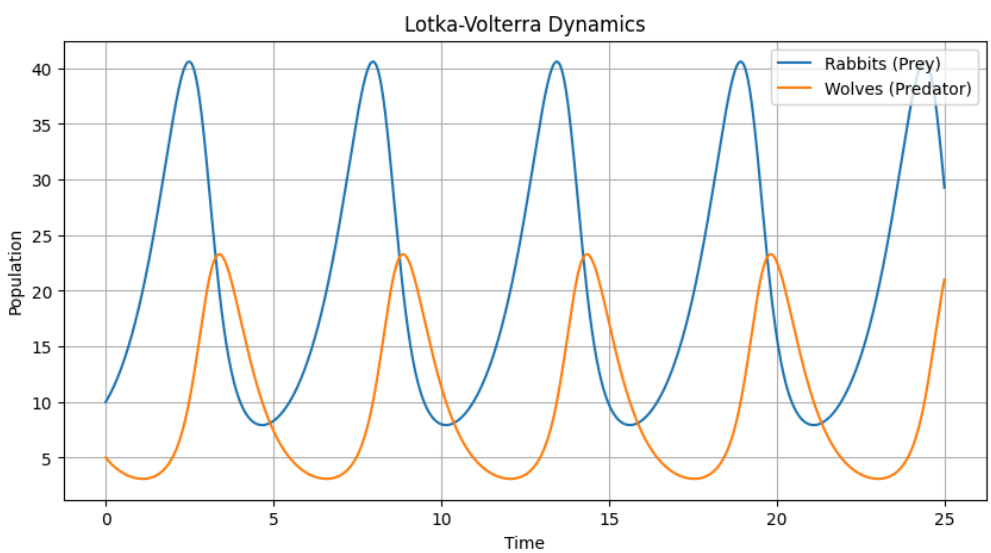
$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

Where:

- (x): prey population (rabbits)
- (y): predator population (wolves)
- (α): prey growth rate
- (β): predation rate
- (γ): predator death rate
- (δ): predator reproduction rate

This creates a **cyclic dynamic system**, where populations rise and fall periodically.



Application of Neural ODE

Instead of explicitly defining the above equations, we use a **Neural ODE** to *learn the dynamics* directly from data.

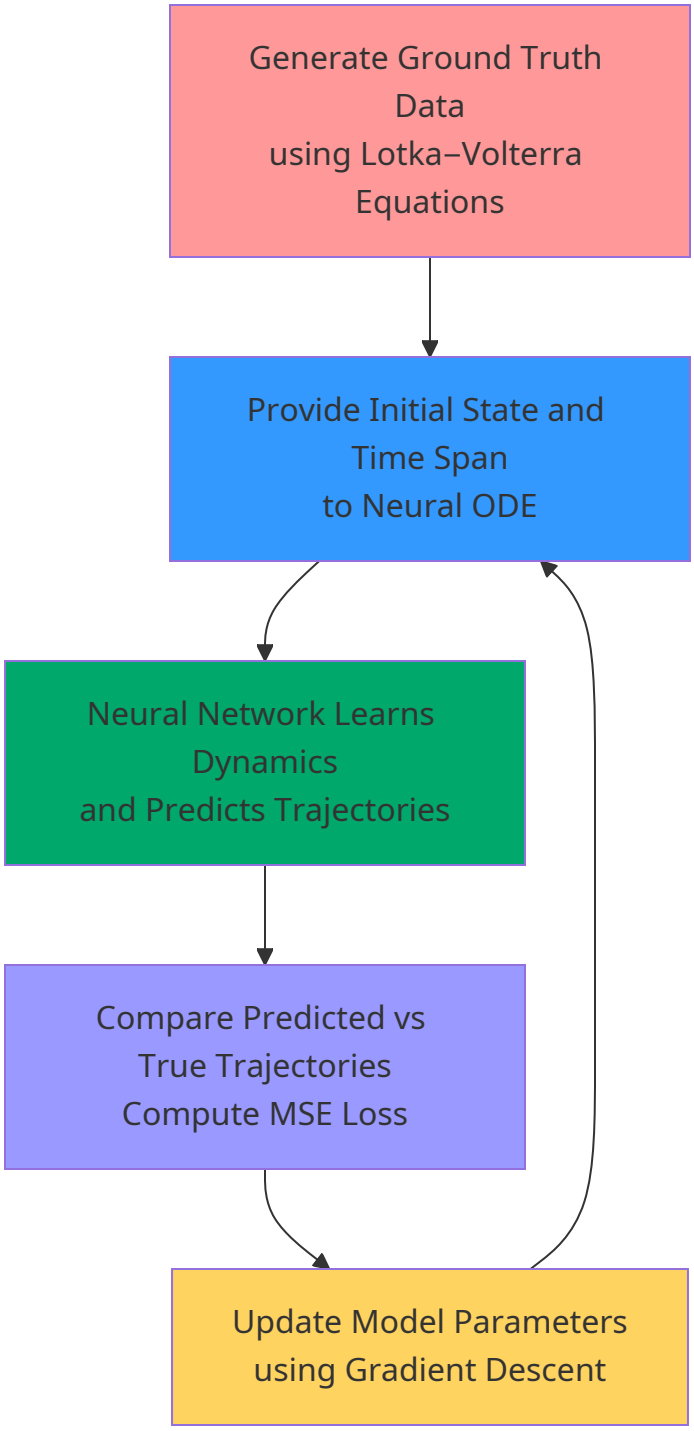
What we are doing:

- Treat the system as a **continuous-time function**
- Replace the analytical derivative function with a **neural network**

- The model learns:

$$\frac{dy}{dt} = f_\theta(y)$$

Workflow

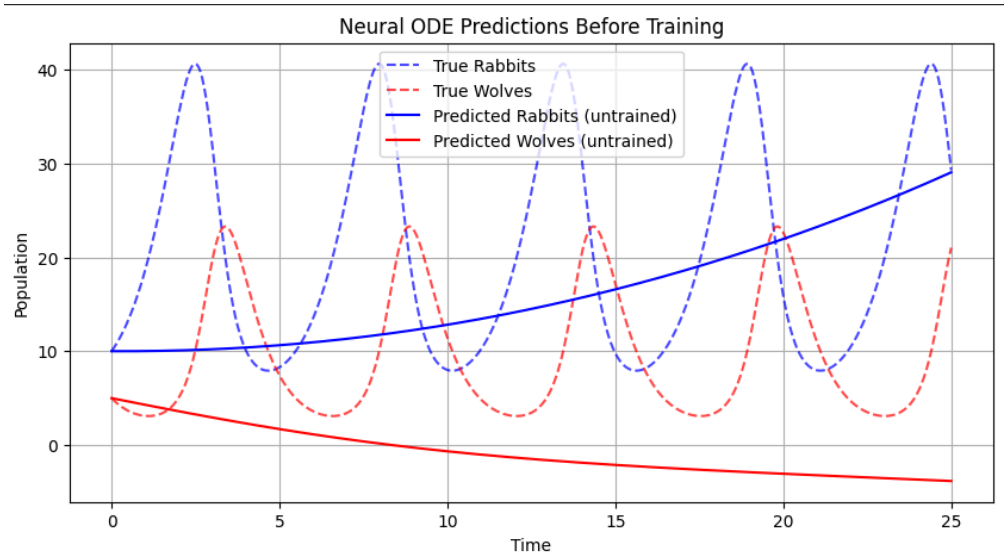


This allows the model to **discover underlying system dynamics** without prior knowledge of equations.

Test on Untrained Model

Before training, the Neural ODE:

- Has **random weights**
- Produces trajectories that **do not match real dynamics**



✓ **Observation**

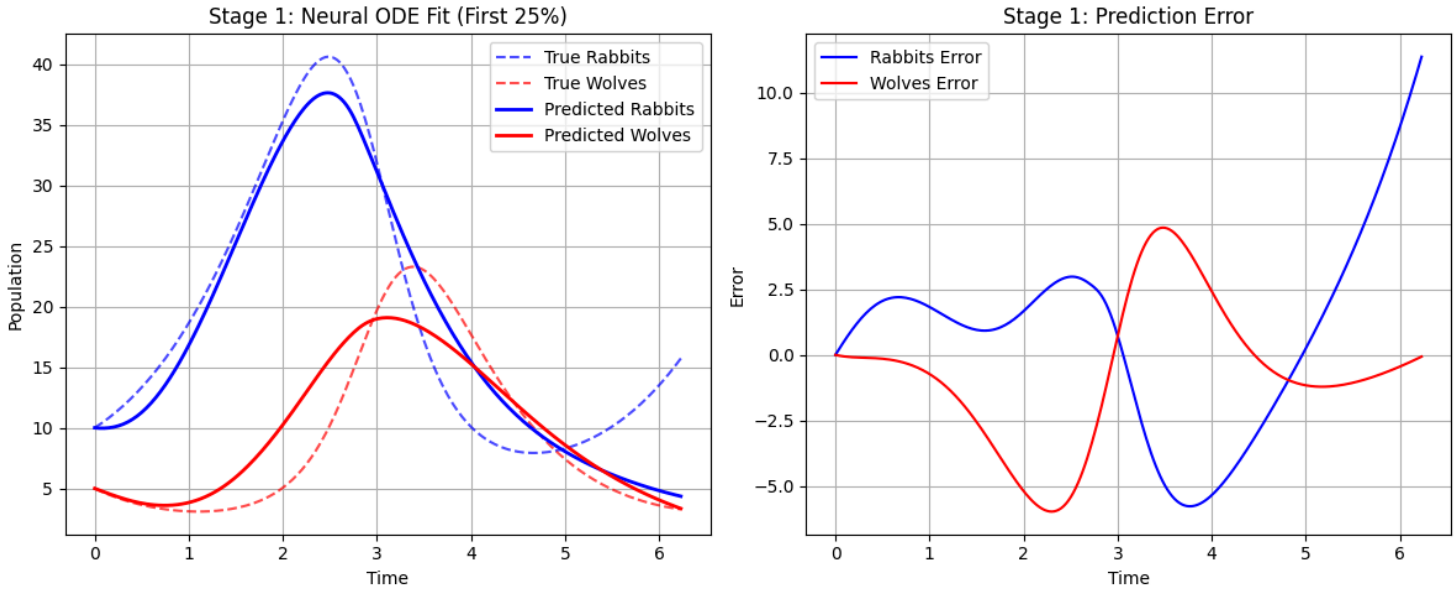
- Predictions are unstable and incorrect
- No clear oscillatory pattern
- Large deviation from true curves

Neural ODE does not "know physics" initially — it must learn from data.

Stage 1 - Test on 25% Trained Model

Params

- Trained on **first 25% of time series**
- 500 epochs
- MSE loss: **11.015**



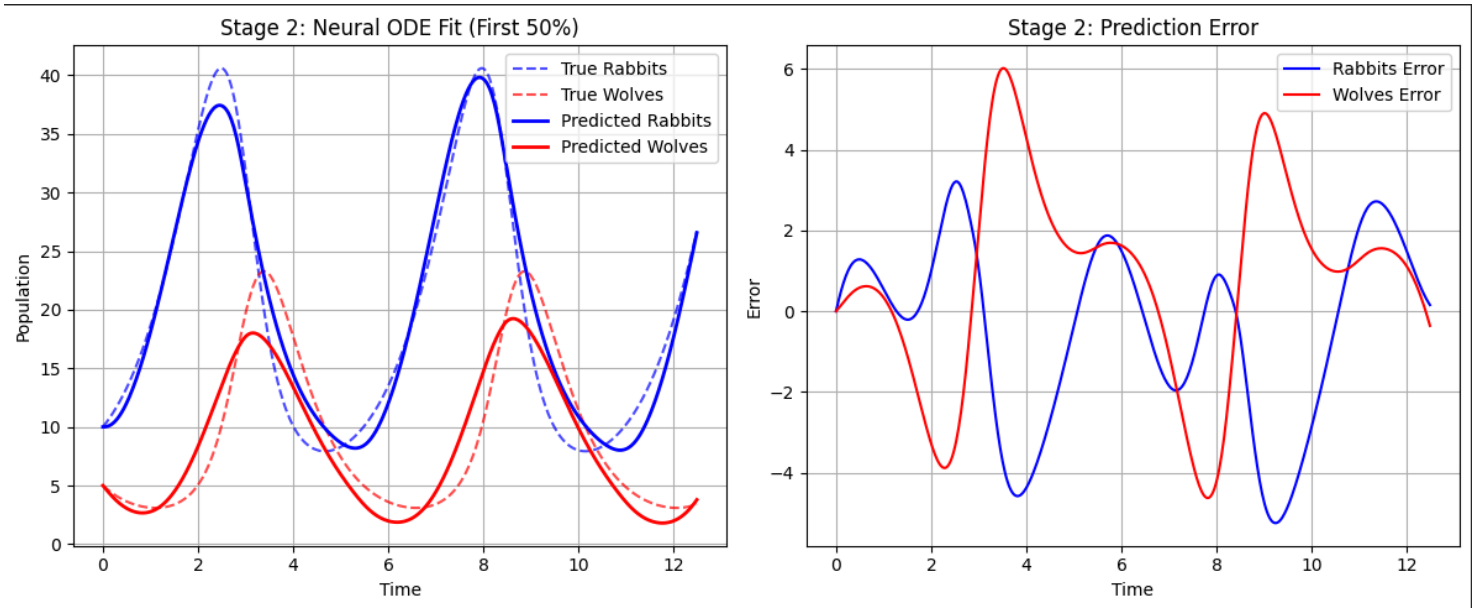
What is happening:

- Model starts capturing **basic trends**
- Still struggles with:
  - amplitude
  - phase alignment

Stage 2 - Test on 50% Trained Model

Params

- Trained on **first 50% of data**
- 500 epochs
- MSE loss: **6.122**



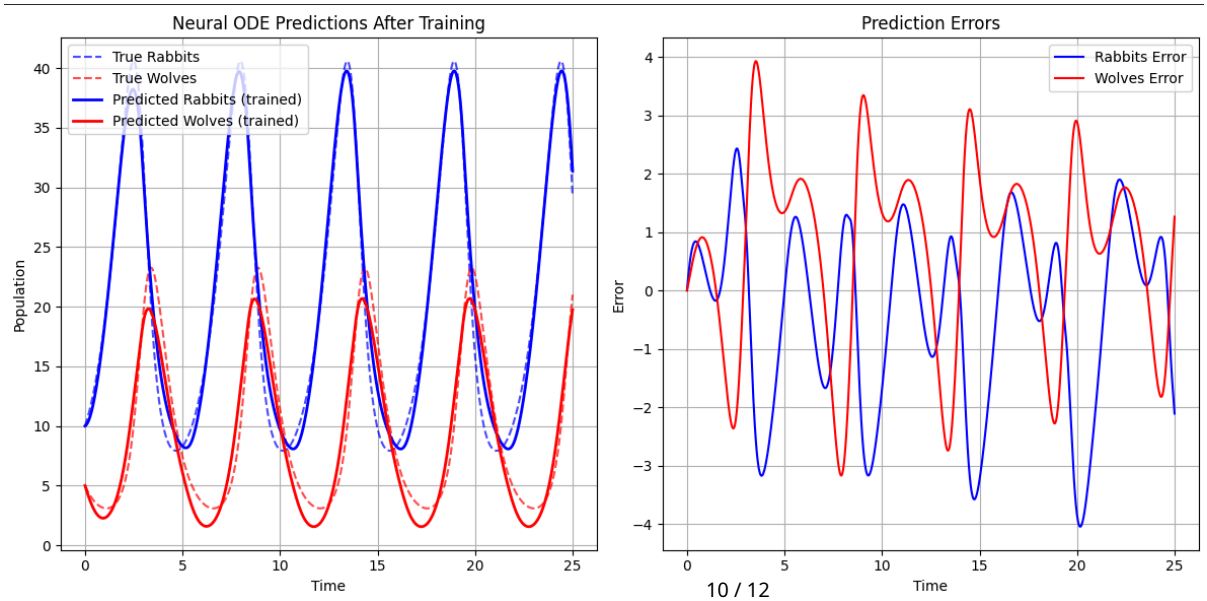
Improvements:

- Better alignment with true trajectories
- Oscillations begin to resemble real system
- Reduced prediction error

Stage 3 - Test on Full Data

Params

- Trained on **entire dataset**
- 500 epochs
- MSE loss: **2.764**

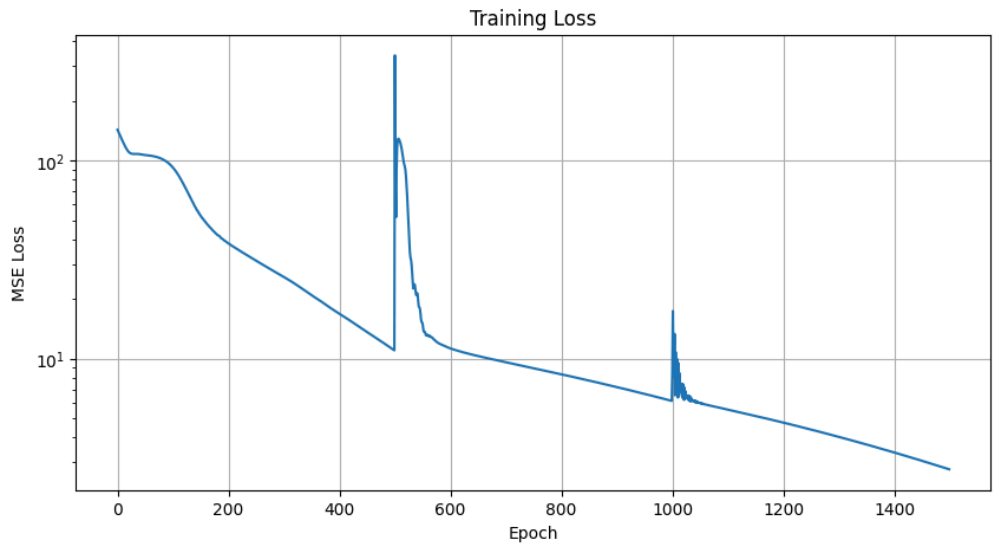


✓ **Observation**

- Predictions closely follow true curves
- Oscillatory dynamics captured well
- Errors significantly reduced

Training Behavior

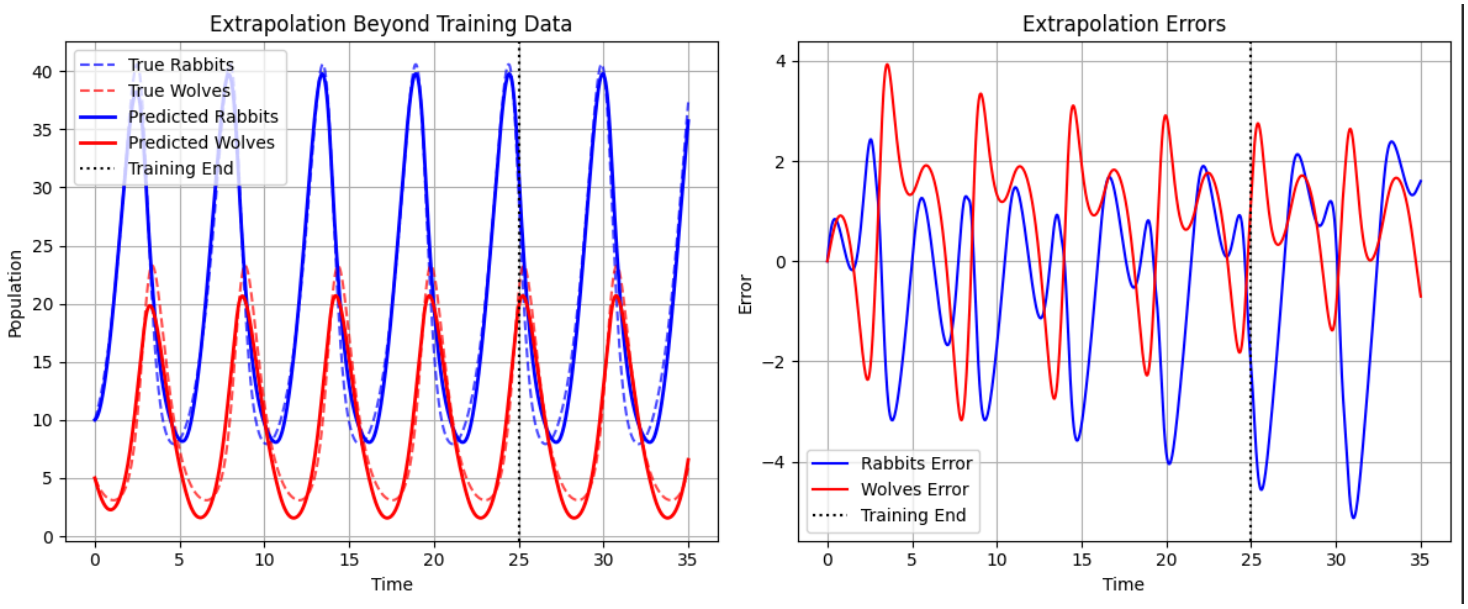
- Loss steadily decreases over time
- Indicates stable learning of dynamics



Extrapolation Test

Now we test beyond training range:

- Training data: $(t = 0 \to 25)$
- Test data: $(t = 0 \to 35)$



✓ Observations

- Model continues oscillatory behavior
- Initially accurate beyond training region
- Errors increase gradually over time

Insight

- Neural ODE learns **general dynamics**, not just data points
- However:
  - Long-term extrapolation is imperfect
  - Small errors accumulate due to recursive integration

Key insight

Neural ODEs are good at interpolation and short-term extrapolation, but long-term predictions can drift.

Code References

See implementation here: `neural_ode.ipynb`